

(1) TensorFlow (Keras)

```
import tensorflow as tf
import numpy as np
```

A. Element-wise addition and multiplication

```
a = tf.constant([34,6,13])
b = tf.constant([12,13,6])

print("Addition: ", tf.add(a,b).numpy())
print("Multiplication: ", tf.multiply(a,b).numpy())
```

```
Addition:  [46 19 19]
Multiplication:  [408  78  78]
```

B. Dense layer with 128 units and ReLU

```
dense_layer = tf.keras.layers.Dense(128, activation='relu')
print(dense_layer)
```

```
<Dense name=dense, built=False>
```

C. Sequential model (256 → 10)

```
model = tf.keras.Sequential([
    tf.keras.layers.Dense(256, activation='relu', input_shape=(100,)),
    tf.keras.layers.Dense(10) # Output layer
])
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input_shape` to `input_shape` of a layer.
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Compile with Adam & Sparse Categorical Crossentropy

```
model.compile(
    optimizer='adam',
    loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
    metrics=['accuracy']
)
```

```
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense_1 (Dense)	(None, 256)	25,856
dense_2 (Dense)	(None, 10)	2,570

Total params: 28,426 (111.04 KB)
Trainable params: 28,426 (111.04 KB)
Non-trainable params: 0 (0.00 B)

Train for 10 epochs (batch size = 32)

```
X_train = np.random.rand(1000, 100)
y_train = np.random.randint(0, 10, size=(1000,))

model.fit(X_train, y_train, epochs=10, batch_size=32)
```

```
Epoch 1/10
32/32 ————— 1s 4ms/step - accuracy: 0.1073 - loss: 2.3319
Epoch 2/10
32/32 ————— 0s 4ms/step - accuracy: 0.1361 - loss: 2.2868
Epoch 3/10
32/32 ————— 0s 4ms/step - accuracy: 0.1666 - loss: 2.2489
Epoch 4/10
32/32 ————— 0s 3ms/step - accuracy: 0.1791 - loss: 2.2173
Epoch 5/10
32/32 ————— 0s 4ms/step - accuracy: 0.2070 - loss: 2.1834
```

```
Epoch 6/10
32/32 ————— 0s 3ms/step - accuracy: 0.2669 - loss: 2.1531
Epoch 7/10
32/32 ————— 0s 4ms/step - accuracy: 0.2745 - loss: 2.1175
Epoch 8/10
32/32 ————— 0s 4ms/step - accuracy: 0.3030 - loss: 2.0763
Epoch 9/10
32/32 ————— 1s 11ms/step - accuracy: 0.3320 - loss: 2.0527
Epoch 10/10
32/32 ————— 0s 10ms/step - accuracy: 0.2955 - loss: 2.0285
<keras.src.callbacks.history.History at 0x7bf9a87eaba0>
```

Save and Load the model

```
model.save("myscore_model.keras")

loaded_model = tf.keras.models.load_model("myscore_model.keras")
loaded_model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense_1 (Dense)	(None, 256)	25,856
dense_2 (Dense)	(None, 10)	2,570

Total params: 85,280 (333.13 KB)
 Trainable params: 28,426 (111.04 KB)
 Non-trainable params: 0 (0.00 B)
 Optimizer params: 56,854 (222.09 KB)

(2) PyTorch

```
import torch
import torch.nn as nn
import torch.optim as optim
```

Element-wise addition and multiplication

```
aa = torch.tensor([34,6,13])
bb = torch.tensor([12,13,6])

print("Addition: ", aa+bb)
print("Multiplication: ", aa*bb)
print('')
print("Addition: ", torch.add(aa,bb))
print("Multiplication: ", torch.mul(aa,bb))
```

```
Addition: tensor([46, 19, 19])
Multiplication: tensor([408, 78, 78])
```

```
Addition: tensor([46, 19, 19])
Multiplication: tensor([408, 78, 78])
```

Dense layer with 128 units and ReLU

```
dense_layer = nn.Linear(in_features=100, out_features=128)
relu = nn.ReLU()

print(dense_layer)
print(relu)
```

```
Linear(in_features=100, out_features=128, bias=True)
ReLU()
```

Sequential model (256 → 10)

```
class SimpleModel(nn.Module):
    def __init__(self):
        super(SimpleModel, self).__init__()
        self.fc1 = nn.Linear(100, 256)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(256, 10)

    def forward(self, x):
```

```

        x = self.relu(self.fc1(x))
        x = self.fc2(x)
        return x

model = SimpleModel()
print(model)

SimpleModel(
  (fc1): Linear(in_features=100, out_features=256, bias=True)
  (relu): ReLU()
  (fc2): Linear(in_features=256, out_features=10, bias=True)
)

```

Loss Function and Optimizer

```

criterion = nn.CrossEntropyLoss() # same as SparseCategoricalCrossentropy(from_logits=True)
optimizer = optim.Adam(model.parameters(), lr=0.001)

```

Train for 10 epochs (batch size = 32)

```

# Random training data (same as TensorFlow example)
X_train = torch.rand(1000, 100)
y_train = torch.randint(0, 10, (1000,))

batch_size = 32
epochs = 10

for epoch in range(epochs):
    permutation = torch.randperm(X_train.size(0))
    correct = 0
    total = 0
    running_loss = 0.0

    for i in range(0, X_train.size(0), batch_size):
        indices = permutation[i:i+batch_size]
        batch_x, batch_y = X_train[indices], y_train[indices]

        optimizer.zero_grad()
        outputs = model(batch_x)
        loss = criterion(outputs, batch_y)
        loss.backward()
        optimizer.step()

        running_loss += loss.item()

    _, predicted = torch.max(outputs.data, 1)
    total += batch_y.size(0)
    correct += (predicted == batch_y).sum().item()

accuracy = correct / total
print(f"Epoch {epoch+1}/{epochs} - Loss: {running_loss:.4f} - Accuracy: {accuracy:.4f}")

```

```

Epoch 1/10 - Loss: 74.2549 - Accuracy: 0.1030
Epoch 2/10 - Loss: 73.3550 - Accuracy: 0.1290
Epoch 3/10 - Loss: 72.6741 - Accuracy: 0.1620
Epoch 4/10 - Loss: 72.1503 - Accuracy: 0.1750
Epoch 5/10 - Loss: 71.3834 - Accuracy: 0.2230
Epoch 6/10 - Loss: 70.6045 - Accuracy: 0.2230
Epoch 7/10 - Loss: 69.7624 - Accuracy: 0.2520
Epoch 8/10 - Loss: 68.7564 - Accuracy: 0.2720
Epoch 9/10 - Loss: 67.9776 - Accuracy: 0.2870
Epoch 10/10 - Loss: 66.6755 - Accuracy: 0.3070

```

Save and Load the Model

```

# Save
torch.save(model.state_dict(), "myscore_model.pth")

# Load
loaded_model = SimpleModel()
loaded_model.load_state_dict(torch.load("myscore_model.pth"))
loaded_model.eval()

print("Model loaded successfully")

Model loaded successfully

```

(3) Define a 2-layer Sequential model

Compile it (loss + optimizer)

Train for 30 epochs

Use data

x = [1,2,3,4], y = [2,4,6,8] (linear model: y = 2x)

print prediction ****bold text****

```
import tensorflow as tf
import numpy as np
```

```
x = np.array([1, 2, 3, 4], dtype=float)
y = np.array([2, 4, 6, 8], dtype=float)
```

```
model = tf.keras.Sequential([
    tf.keras.layers.Dense(8, activation='relu', input_shape=(1,)),
    tf.keras.layers.Dense(1)
])

model.summary()
```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input\_shape`/`input`
super().__init__(activity_regularizer=activity_regularizer, **kwargs)

Model: "sequential_1"

Layer (type)	Output Shape	Param #
dense_3 (Dense)	(None, 8)	16
dense_4 (Dense)	(None, 1)	9

Total params: 25 (100.00 B)

Trainable params: 25 (100.00 B)

Non-trainable params: 0 (0.00 B)

```
model.compile(
    optimizer='sgd',
    loss='mse'
)
```

```
model.fit(x, y, epochs=30, verbose=1)
```

```
1/1 ————— 0s 56ms/step - loss: 29.0100
Epoch 3/30
1/1 ————— 0s 56ms/step - loss: 28.0592
Epoch 4/30
1/1 ————— 0s 53ms/step - loss: 27.1461
Epoch 5/30
1/1 ————— 0s 54ms/step - loss: 26.2691
Epoch 6/30
1/1 ————— 0s 61ms/step - loss: 25.4268
Epoch 7/30
1/1 ————— 0s 61ms/step - loss: 24.6179
Epoch 8/30
1/1 ————— 0s 51ms/step - loss: 23.8410
Epoch 9/30
1/1 ————— 0s 55ms/step - loss: 23.0949
Epoch 10/30
1/1 ————— 0s 58ms/step - loss: 22.3784
Epoch 11/30
1/1 ————— 0s 52ms/step - loss: 21.6902
Epoch 12/30
1/1 ————— 0s 68ms/step - loss: 21.0293
Epoch 13/30
1/1 ————— 0s 48ms/step - loss: 20.3945
Epoch 14/30
1/1 ————— 0s 55ms/step - loss: 19.7849
Epoch 15/30
1/1 ————— 0s 56ms/step - loss: 19.1994
```

```
1/1 ————— 0s 53ms/step - loss: 18.0971
Epoch 18/30
1/1 ————— 0s 49ms/step - loss: 17.5784
Epoch 19/30
1/1 ————— 0s 54ms/step - loss: 17.0803
Epoch 20/30
1/1 ————— 0s 53ms/step - loss: 16.6019
Epoch 21/30
1/1 ————— 0s 49ms/step - loss: 16.1425
Epoch 22/30
1/1 ————— 0s 49ms/step - loss: 15.7013
Epoch 23/30
1/1 ————— 0s 50ms/step - loss: 15.2775
Epoch 24/30
1/1 ————— 0s 56ms/step - loss: 14.8705
Epoch 25/30
1/1 ————— 0s 50ms/step - loss: 14.4796
Epoch 26/30
1/1 ————— 0s 54ms/step - loss: 14.1042
Epoch 27/30
1/1 ————— 0s 51ms/step - loss: 13.7437
Epoch 28/30
1/1 ————— 0s 50ms/step - loss: 13.3975
Epoch 29/30
1/1 ————— 0s 56ms/step - loss: 13.0649
Epoch 30/30
1/1 ————— 0s 56ms/step - loss: 12.7456
```

```
test_x = np.array([5, 6, 7], dtype=float)
predictions = model.predict(test_x)

print("Predictions:")
for i in range(len(test_x)):
    print(f"x = {test_x[i]} → y ≈ {predictions[i][0]:.2f}")
```

```
1/1 ————— 0s 231ms/step
Predictions:
x = 5.0 → y ≈ 2.27
x = 6.0 → y ≈ 2.27
x = 7.0 → y ≈ 2.27
```